

Foundations of Self-supervised learning

[Self-supervised learning]

1.0 Content Block

Pseudo-labels Pseudo-labels are automatically generated labels that a model assigns to unlabeled data based on its own predictions. They are widely used in self-supervised and semi-supervised learning, where ground-truth annotations are limited or unavailable. By treating predicted labels as surrogate ground truth, learning algorithms can make use of large quantities of unlabeled data in the training process. Pseudo-labeling also plays an important role in systems that must adapt to concept drift, where the statistical properties of the data change over time. In these scenarios, the model may detect that an incoming instance deviates from previously learned behavior. The system then generates a classification result for that instance, and this predicted class is used as a pseudo-label for updating or retraining model components that are becoming outdated. This approach enables continuous adaptation in dynamic environments without requiring manual annotation. In many adaptive learning pipelines, pseudo-labels are chosen when the classifier produces sufficiently confident predictions, reducing the risk of propagating errors. These pseudo-labeled instances are then incorporated into training to refresh or evolve the model's understanding of emerging data patterns, particularly when existing components show signs of "aging" due to drift or distributional shifts. This strategy reduces reliance on manual labeling while helping maintain long-term model performance.

2.0 Content Block

$$\mathcal{L}_N = -\mathbb{E}_{X \sim p} \left[\log \left(\sum_{j \in \mathcal{X}} f_k(x_j, c_t) \right) \right] \quad \{\text{displaystyle } \{\mathcal{L}\}_{\mathrm{N}}\}$$

$$= -\mathbb{E}_{X \sim p} \left[\log \left(\frac{f_k(x_{t+k}, c_t)}{\sum_{j \in \mathcal{X}} f_k(x_j, c_t)} \right) \right]$$

3.0 Content Block

Self-supervised learning (SSL) is a paradigm in machine learning where a model is trained on a task using the data itself to generate supervisory signals, rather than relying on externally-provided labels. In the context of neural networks, self-supervised learning aims to leverage inherent structures or relationships within the input data to create meaningful training signals. SSL tasks are designed so that solving them requires capturing essential features or relationships in the data. The input data is typically augmented or transformed in a way that creates pairs of related samples, where one sample serves as the input, and the other is used to formulate the supervisory signal. This augmentation can involve introducing noise, cropping, rotation, or other transformations. Self-supervised learning more closely imitates the way humans learn to classify objects. During SSL, the model learns in two steps. First, the task is solved based on an auxiliary or pretext classification task using pseudo-labels, which help to initialize the model parameters. Next, the actual task is performed with supervised or unsupervised learning. Self-supervised learning has produced promising results in recent years, and has found practical application in fields such as audio processing, and is being used by Facebook and others for speech recognition.

4.0 Content Block

Comparison with other forms of machine learning SSL belongs to supervised learning methods insofar as the goal is to generate a classified output from the input. At the same time, however, it does not require the explicit use of labeled input-output pairs. Instead, correlations, metadata embedded in the data, or domain knowledge present in the input are implicitly and autonomously extracted from the data. These supervisory signals, extracted from the data, can then be used for training. SSL is similar to unsupervised learning in that it does not require labels in the sample data. Unlike unsupervised learning, however, learning is not

done using inherent data structures. Semi-supervised learning combines supervised and unsupervised learning, requiring only a small portion of the learning data be labeled. In transfer learning, a model designed for one task is reused on a different task. Training an autoencoder intrinsically constitutes a self-supervised process, because the output pattern needs to become an optimal reconstruction of the input pattern itself. However, in current jargon, the term 'self-supervised' often refers to tasks based on a pretext-task training setup. This involves the (human) design of such pretext task(s), unlike the case of fully self-contained autoencoder training. In reinforcement learning, self-supervised learning from a combination of losses can create abstract representations where only the most important information about the state are kept in a compressed way.

5.0 Content Block

Non-contrastive self-supervised learning Non-contrastive self-supervised learning (NCSSL) uses only positive examples. Counterintuitively, NCSSL converges on a useful local minimum rather than reaching a trivial solution, with zero loss. For the example of binary classification, it would trivially learn to classify each example as positive. Effective NCSSL requires an extra predictor on the online side that does not back-propagate on the target side.

6.0 Content Block

Contrastive self-supervised learning For a binary classification task, training data can be divided into positive examples and negative examples. Positive examples are those that match the target. For example, if training a classifier to identify birds, the positive training data would include images that contain birds. Negative examples would be images that do not. Contrastive self-supervised learning uses both positive and negative examples. The loss function in contrastive learning is used to minimize the distance between positive sample pairs, while maximizing the distance between negative sample pairs. An early example uses a pair of 1-dimensional convolutional neural networks to process a pair of images and maximize their agreement. Contrastive Language-Image Pre-training (CLIP) allows joint pretraining of a text encoder and an image encoder, such that a matching image-text pair have image encoding vector and text encoding vector that span a small angle (having a large cosine similarity). InfoNCE (Noise-Contrastive Estimation) is a method to optimize two models jointly, based on Noise Contrastive Estimation (NCE). Given a set $X = \{x_1, \dots, x_N\}$ of N random samples containing one positive sample from $p(x_{t+k} | c_t)$ and $N - 1$ negative samples from the 'proposal' distribution $p(x_{t+k})$, it minimizes the following loss function:

7.0 Content Block

Joint-Embedding and Predictive Architectures A major class of self-supervised learning moves beyond contrastive pairs, instead maximizing the agreement between views while preventing collapse through statistical constraints. Rooted in Deep Canonical Correlation Analysis (Deep CCA), this approach includes Joint-Embedding Architectures (JEA) like Barlow Twins and VICReg, which enforce covariance constraints to learn invariant representations without negative sampling. Deep Latent Variable Path Modelling (DLVPM) generalizes this to multimodal systems, using path models to enforce correlation and orthogonality across diverse data types. In 2022, this framework evolved into Joint-Embedding Predictive Architectures (JEPA) when Yann LeCun introduced JEPA as a step towards decision making, reasoning, and autonomous human intelligence in machines, including self-improvement through autonomous learning. Founded in representation learning, LeCun established the concept of a "world model" in JEPA which aims to enable machines to replicate human intellect by providing machines with a concept for the world in which they exist. Unlike autoencoders, JEPAs operate entirely in latent space, avoiding pixel-level noise to focus on semantic structure, a key step toward autonomous world models. Rather than just learning invariance, JEPAs learn by predicting masked latent representations from visible context. Recently, there have been several new applications of JEPA for things like image analysis, audio

processing, and even motion in images and video.